2026

Working with Database Data in SAS® Viya®

Mark Jordan, SAS Jedi (Retired)

ABSTRACT

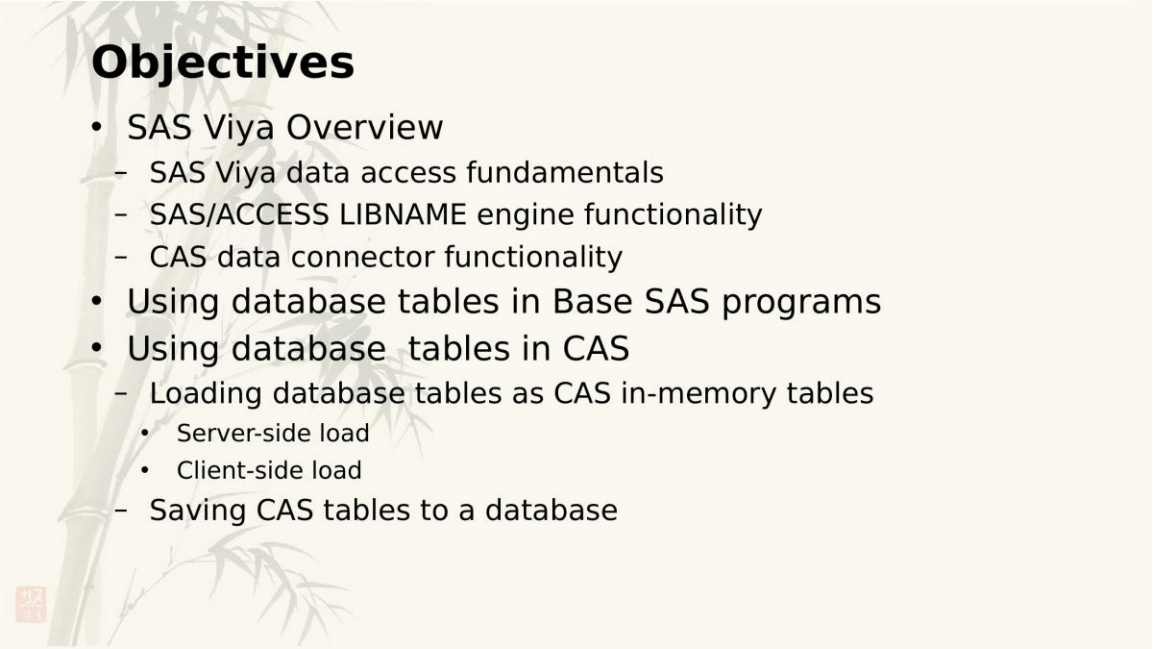
Traditional SAS programming skills remain valuable in SAS Viya, but getting the most from enterprise databases means understanding both the SAS/ACCESS LIBNAME engine and the new, high-performance CAS data connectors. This paper compares every major technique side-by-side: implicit and explicit SQL pass-through, server-side versus client-side CAS loads, and saving CAS in-memory tables back to databases. You'll get a clear decision matrix, real performance benchmarks, and copy-paste code patterns to maximize in-database execution and move data efficiently between your database, SAS Compute, and CAS. Whether you're on SAS 9 or fully in Viya, you'll leave ready to make informed choices that reduce data movement and speed up analytics.

INTRODUCTION

In this paper I'll discuss and demonstrate:

- A brief introduction to SAS/ACCESS Interface technology
- A brief introduction to CAS and SAS Viya data connector technology
- Efficient use of the SAS/ACCESS Interfaces to access DBMS tables in SAS 9 and on the SAS Viya Compute server, including:
    - Transparent data access using the LIBNAME engine (implicit pass-through)
    - Executing your own database-specific SQL from SS (explicit pass-through)
- Using both PROC FedSQL and PROC DS2 to access DBMS tables to manipulate most ANSI SQL data types at full precision in SAS 9 and on the SAS Viya Compute server, including:
    - Implicit pass-through in FedSQL and DS2
    - Explicit pass-through in FedSQL
    - Using FedSQL on a DS2 data program SET statement for explicit pass-through.
- Efficient access and use of DBMS tables using caslibs in CAS, including:
    - Implicit pass-through with FedSQL
    - Explicit pass-through

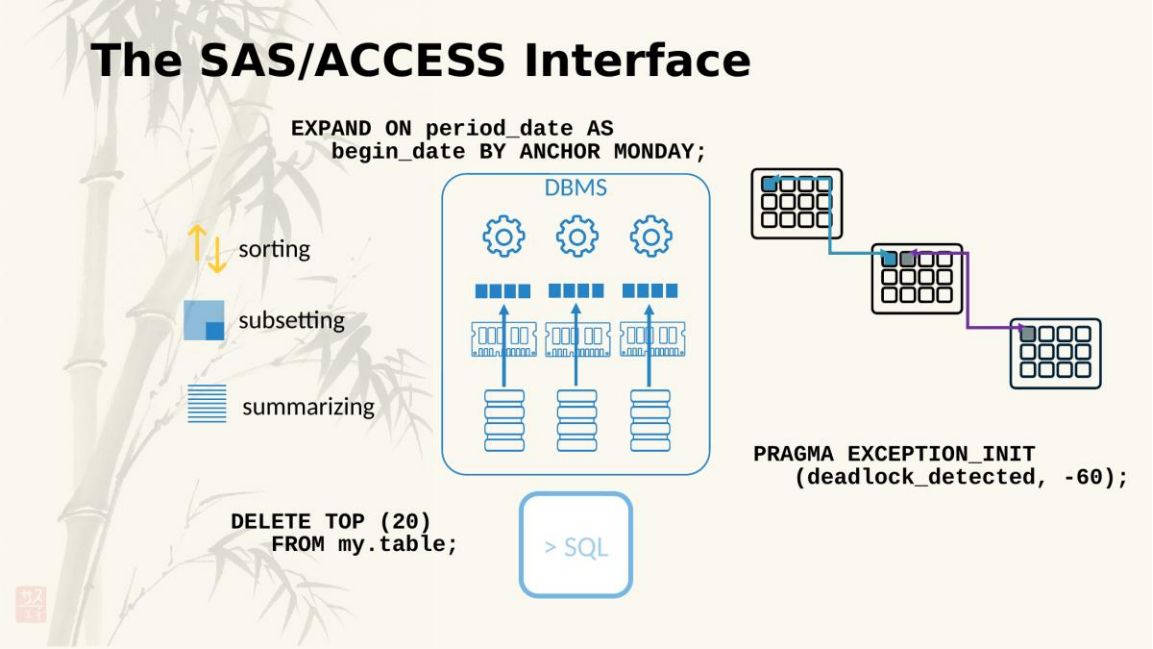
OBJECTIVES



Today I'm hoping to reinforce the fact that in SAS Viya your hard-won SAS skills are still very useful. But SAS Viya includes something new - Cloud Analytic Services, or CAS, as I prefer to call it. CAS can provide extra speed and functionality when working with database data. First, we'll do a quick overview of SAS Viya, comparing accessing data on the Compute Server and in CAS. Next, we'll demonstrate that the techniques we've used to access database data in SAS 9 are still valid on the SAS Viya Compute Server. Then we'll compare different techniques for loading database data into CAS. And finally, we'll demonstrate how to save CAS data to a database table.

SAS9 AND THE SAS VIYA COMPUTE SERVER

THE SAS/ACCESS INTERFACE

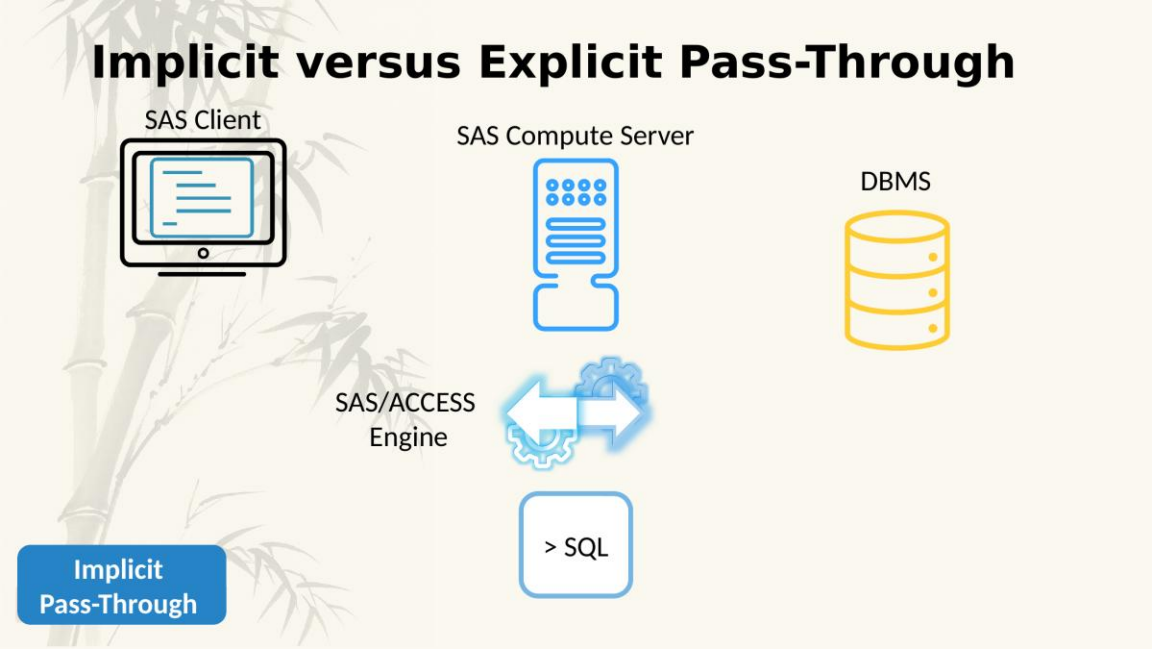


Databases combine hardware and software into a system well equipped for sorting, subsetting, and summarizing data from normalized, tabular data stores. Tables are stored distributed across multiple disks so that data can be retrieved in parallel, maximizing I/O speed. The database is usually well provisionedwith lots of RAM and CPU for snappy response to ad-hoc queries. The native language of most databases is the Structured Query Language, or SQL, with each database speaking its own special dialect. SAS/ACCESS Interfaces enable SAS programmers to leverage the processing power of the database in their SAS programs without having to learn the database's SQL.

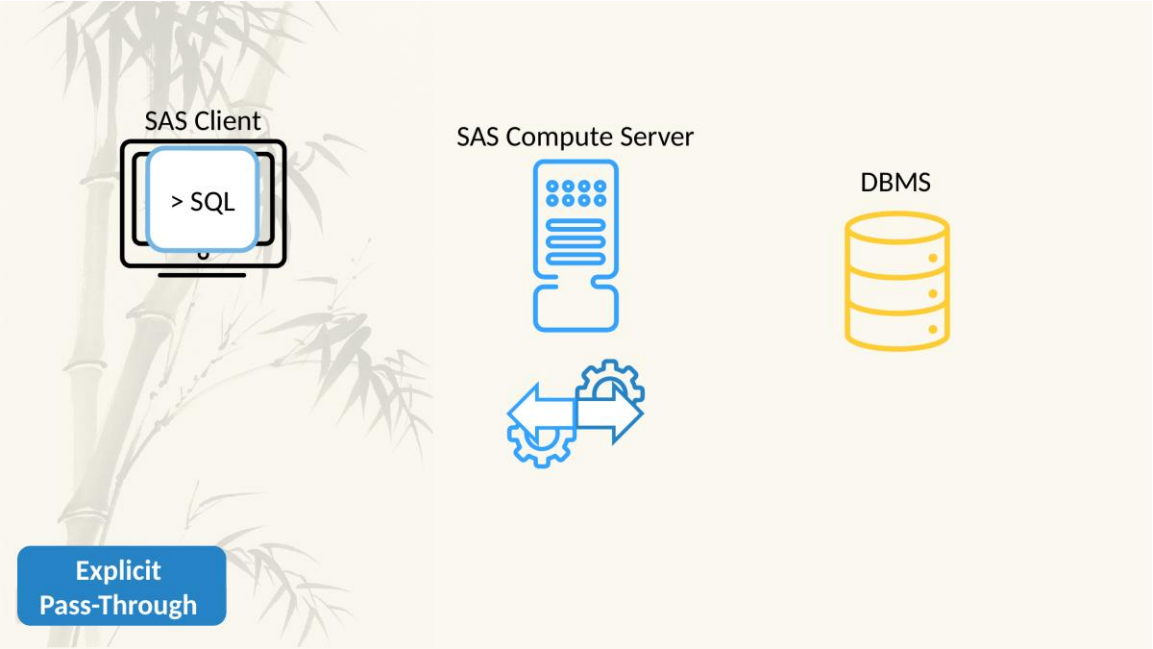


In both SAS 9 and SAS Viya, SAS/ACCESS software on the Compute Server enables two-way interaction with databases from SAS programs. SAS/ACCESS has two primary functions. First, two-way data type translation ensures that SAS receives only double-precision floating-point numeric and fixed-width character values when reading from database tables, and that the database receives appropriate datatypes when data is written back to the database. Second, SAS WHERE statement expressions are translated into database-specific SQL so that the database can subset the data before it is delivered to

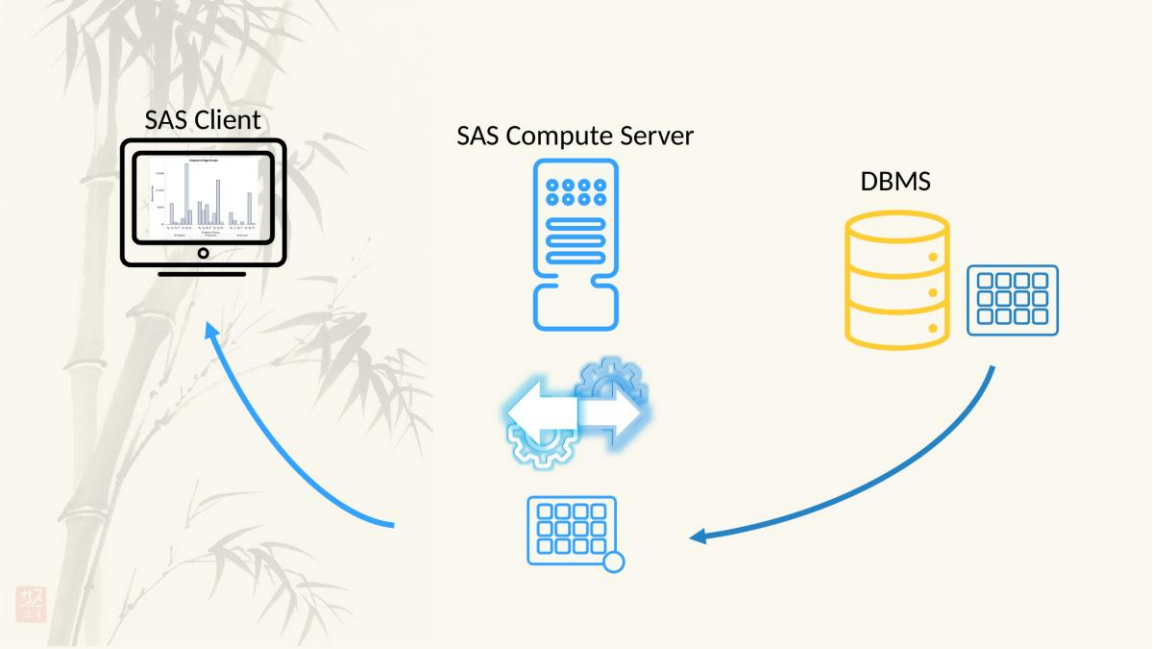
SAS for processing. Processing the WHERE clause in the database significantly reduces overall processing time.



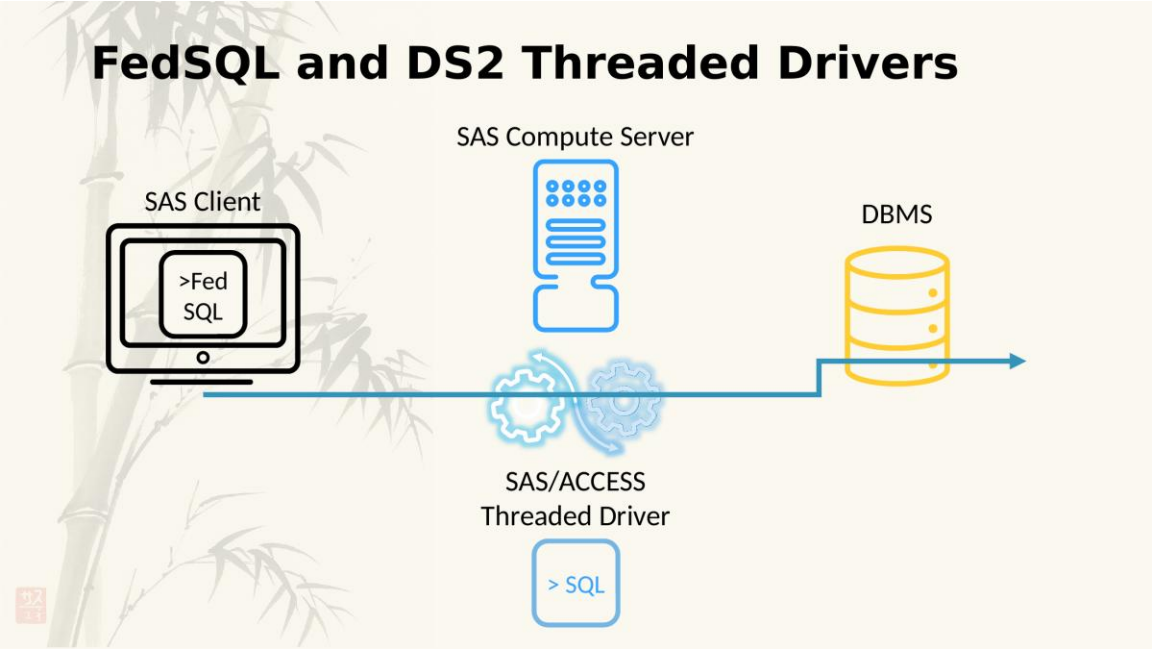
The SAS/Access Interface provides two modes of communicating with the database.The SAS/ACCESS LIBNAME engine provides transparent access to the database so that you can use database tables in your SAS code just as if they were SAS datasets in traditional SAS libraries. You can just write and submit your SAS code, and the LIBNAME engine handles communication with the database, translating as much of your code as possible into database SQL for execution in the database. We call this method "implicit pass-through" because we don't write the SQL passed to the database ourselves.



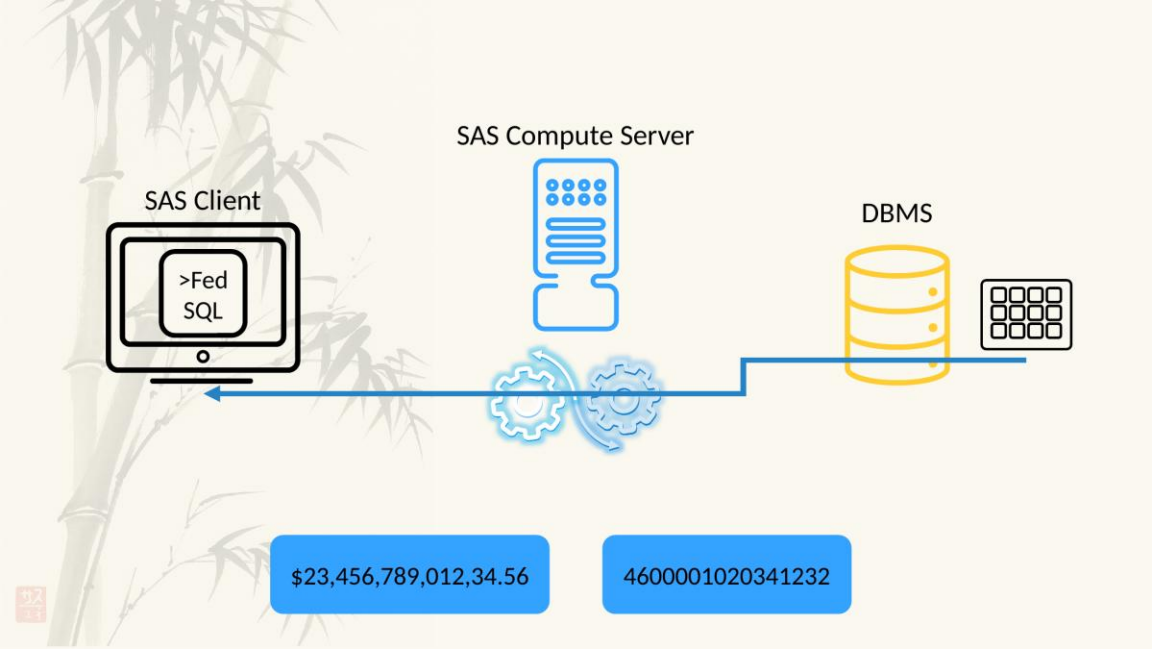
But perhaps you have some pre-prepared database SQL and just want to execute it from your SAS program. You can use special PROC SQL syntax to send that code directly to the database without modification or syntax checking by SAS. Because we write the database SQL ourselves, we refer to this as "explicit pass-through".



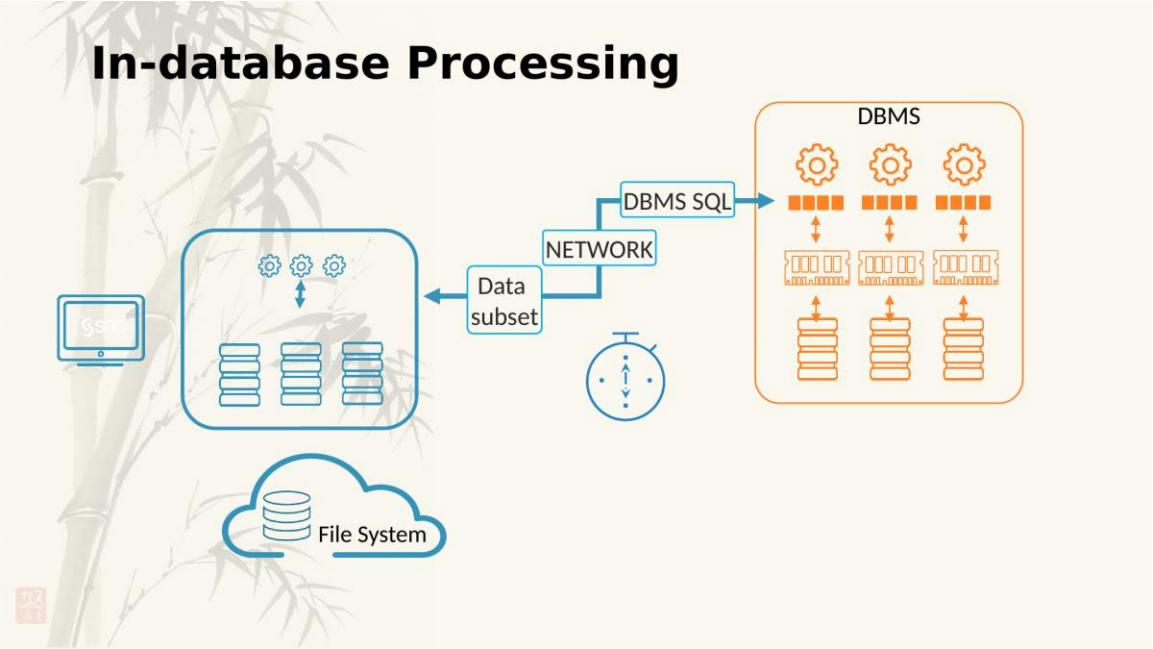
Regardless of how the SQL was generated, when the database finishes processing the SQL, any result set is returned to the SAS/ACCESS interface, which transforms all database data types to SAS data types before the data is sent back to your SAS program for whatever additional processing you have specified.



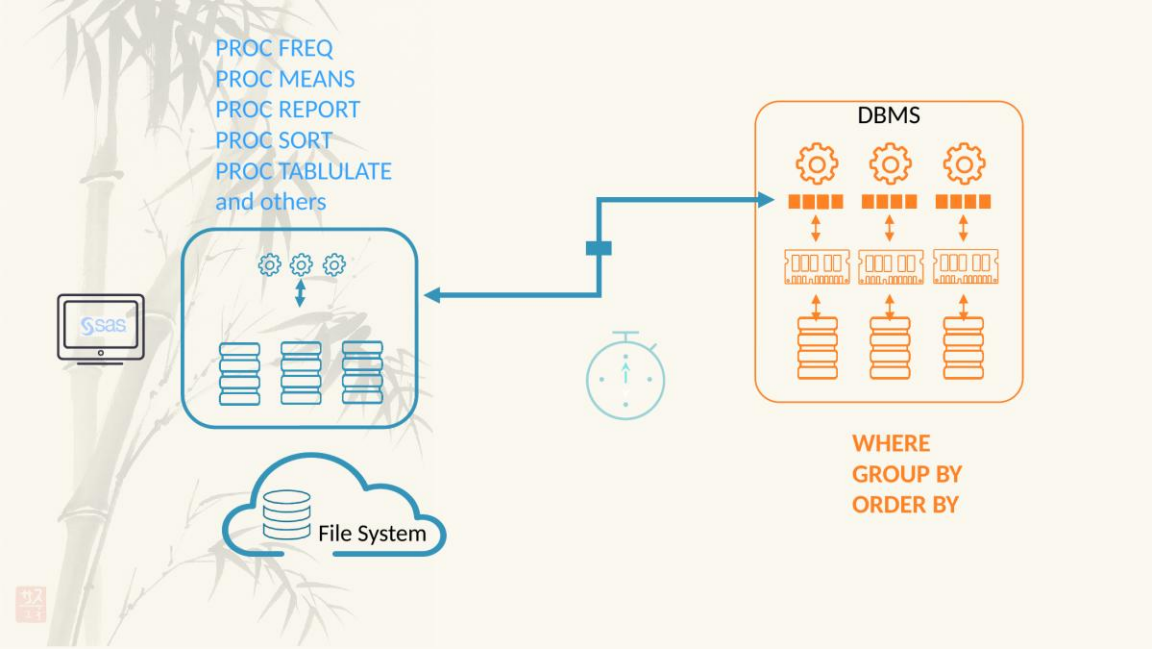
Newer SAS languages, like FedSQL and DS2, can handle most ANSI data types so they don't need the SAS/ACCESS LIBNAME engine's data translation. For supported databases, the SAS/ACCESS interface ships with a separate threaded driver which supports both implicit and explicit pass-through for FedSQL. When code is submitted, the driver once again pushes as much of the SQL as possible into the database.



But after the database processes the SQL, the result set is sent back to your SAS program without modification. This allows DS2 and FedSQL to process high-precision numeric data types, like DECIMAL and BIGINT, without any loss of precision.



The availability of in-database processing provides an opportunity to further reduce the computational burden on SAS by offloading most subsetting, summarization, and sorting tasks to the database using machine-generated native SQL. The database is specifically tuned for this type of work and is generally well equipped with lots of RAM, fast disks, and multiple CPUs. In most cases, the data returned to SAS for processing is much smaller than the original table, so fewer rows are processed by SAS. Of course, this introduces network latency, but the effect is usually overshadowed by increased processing speed due to parallel processing in the database and reduced data movement, so the overall process runs faster.



Many SAS procedures have been enhanced so that when data is sourced from a database table, complex SQL is generated to ensure subsetting, summarization, and sorting take place in the database. The summarized data returned to SAS for processing is generally quite smallso the overall process runs very fast.

DEMO: Implicit Pass-through in SAS9 and the Compute Server

```
/* A quick introduction to implicit pass-through */
options sastrace=',,,d' sastraceloc=saslog nostsuffix;
Title "PROC PRINT results - Implicit Pass-through";
Title2 "Trucks not built in the USA";
proc print data=mydb.Cars;
    var Origin Type MSRP;
    where Type='Truck' and Origin <> 'USA';
run;

Title "PROC FREQ results - Implicit Pass-through";
Title2 "Type of vehicle by origin";
proc freq data=mydb.Cars;
    tables Type*Origin /nopercent nocol norow;
run;

Title "PROC MEANS results - Implicit Pass-through";
Title2 "Highway mileage by origin";
proc means data=mydb.Cars;
    class origin;
    var MPG_Highway;
run;

Title "PROC SQL results - Implicit Pass-through";
Title2 "Number of models by origin";
proc sql;
select Origin, count(*) as Models
    from mydb.Cars
    group by Origin
```

```
    order by 2 desc, 1
;
quit;

Title "PROC FedSQL results - Implicit Pass-through";
Title2 "Number of models by origin";
proc FedSQL;
select Origin, count(*) as Models
    from mydb."Cars"
    group by Origin
    order by count(*) desc, Origin
;
quit;
title;
```

DEMO: Explicit Pass-through in SAS9 and the Compute Server

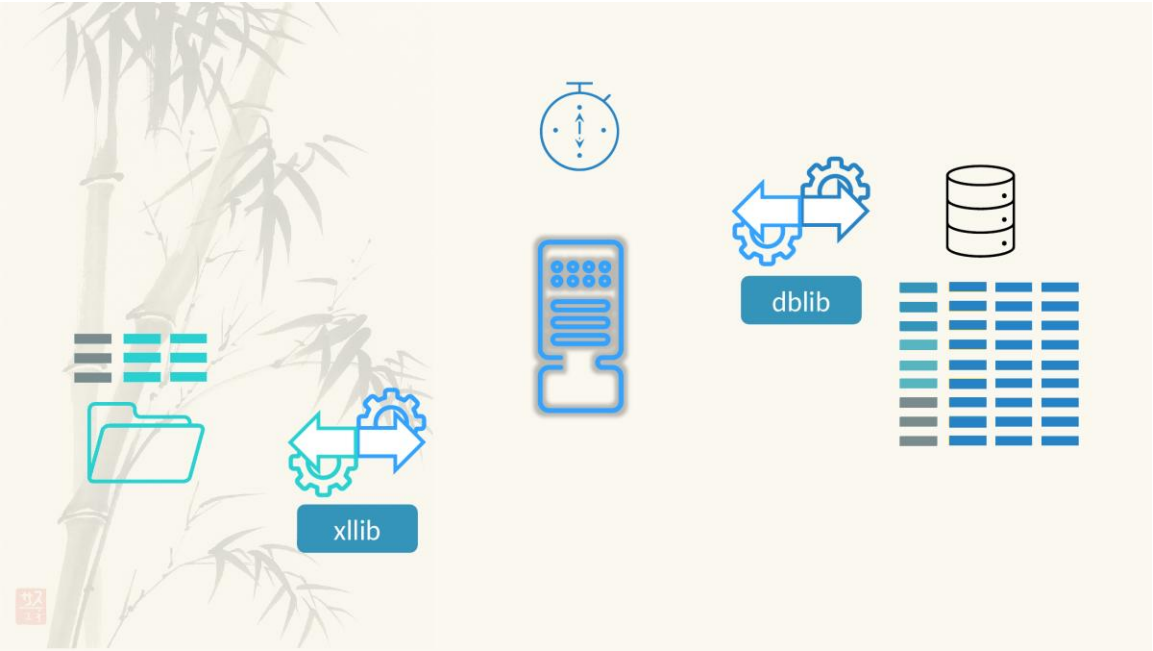
```
/* A quick introduction to explicit pass-through - PROC SQL */
title "PROC SQL - Explicit Pass-through Results";
title2 "Select * from Cars";
proc sql;
    connect using mydb;
    select * from connection to mydb
          (select * from wwdv4.Cars limit 5);
title2 "MySQL JSON_ARRAY function";
    select * from connection to mydb
          (select convert(JSON_ARRAY(Make, Model, Type, MSRP, MPG_Highway
), CHAR(100))
                as "JSON [Make, Model, Type, MSRP, MPG]"
                from wwdv4.Cars limit 5);
    disconnect from mydb;
quit;

/* PROC FedSQL explicit pass-through. Use libref, no connect and disconnect
statement  */
title "PROC FedSQL - Explicit Pass-through Results";
title2 "Select * from Cars";
proc fedsql;
    select * from connection to mydb
          (select * from wwdv4.Cars limit 5);
title2 "MySQL JSON_ARRAY function";
    select * from connection to mydb
          (select convert(JSON_ARRAY(Make, Model, Type, MSRP, MPG_Highway
), CHAR(100))
                as "JSON [Make, Model, Type, MSRP, MPG]"
                from wwdv4.Cars limit 5);
quit;
```

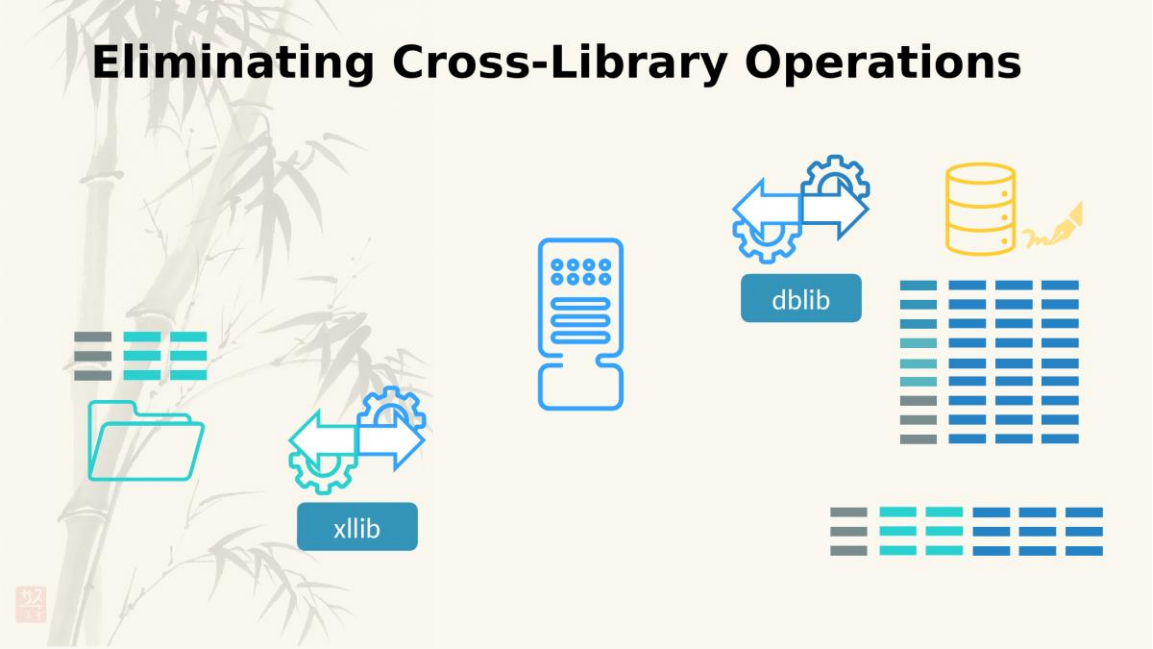
CROSS-LIBRARY OPERATIONS



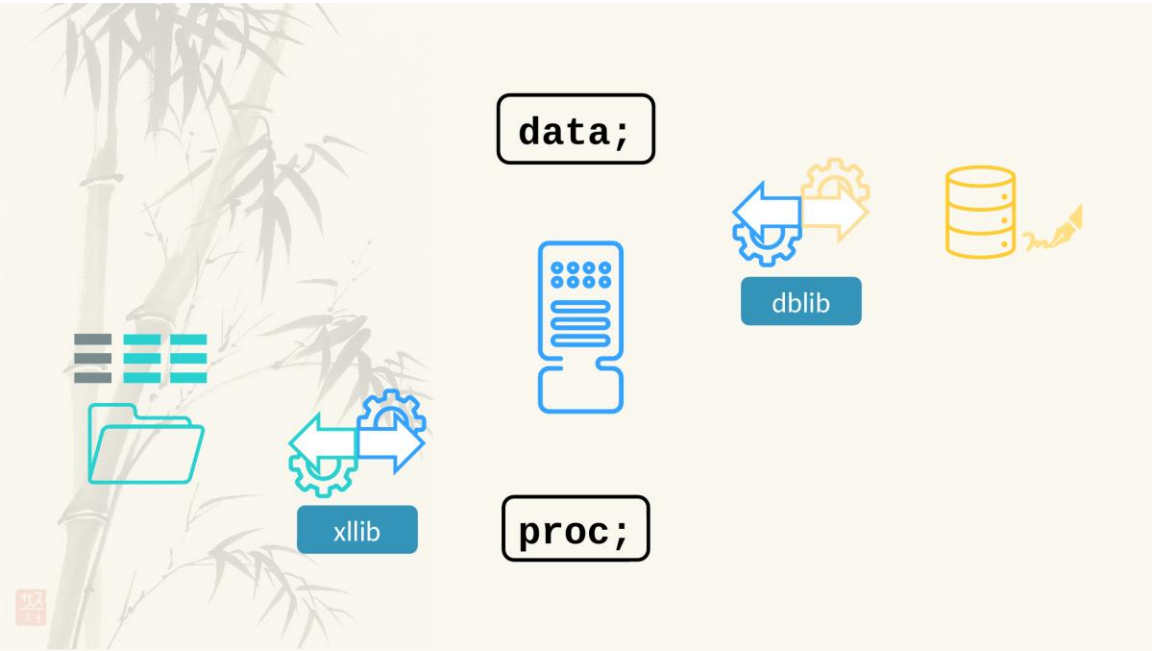
SAS can access a variety of data sources like databases, path-based files, and cloud data using LIBNAME engines. With these engines, SAS provides a common, unifying platform where disparate data types can be processed together without having to learn several additional languages and variations.



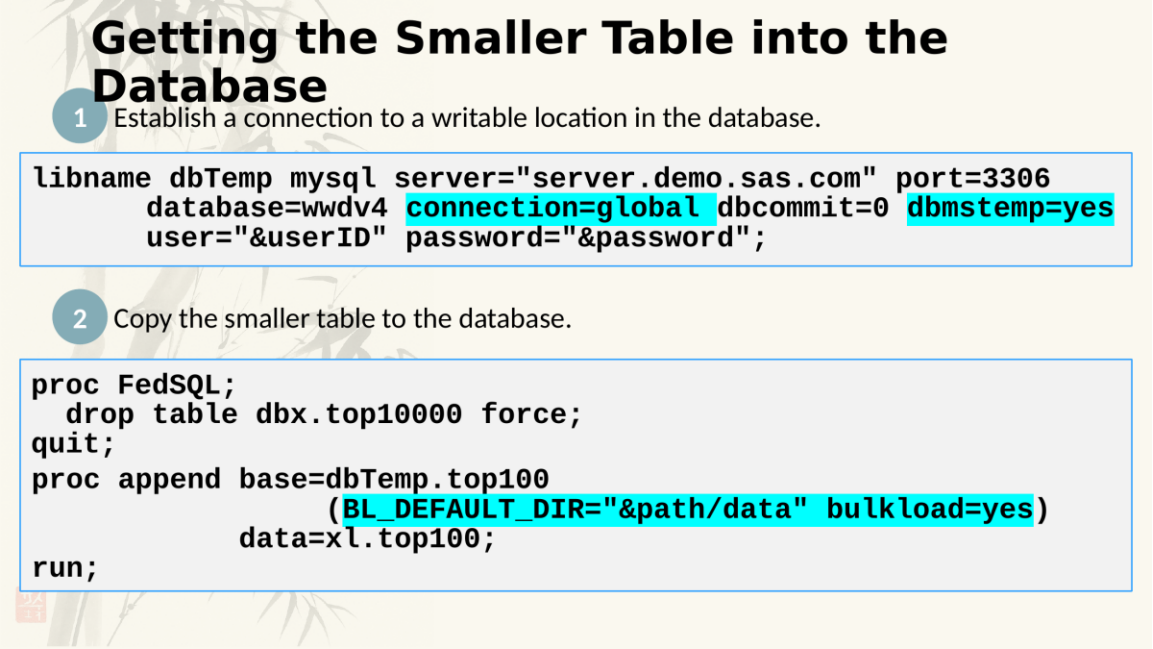
To be processed by SAS, data must first be made available in RAM on the Compute Server. For example, you might wish to join an Excel spreadsheet and a database table. If your result does not require all columns, SAS/ACCESS might eliminate unnecessary columns before transferring the data, but all rows from both sources must be moved into SAS to properly process the join. The larger the database table, the more latency due to network transfer will be experienced.



The most effective way to maximize in-database processing is to get all the data you need into the database. The steps involved include copying the smaller table into the database, performing the join, returning the result to SAS, and then dropping the copy of the small table. For this technique to work, you will need write access to a database schema. Most databases provide write access a personal schema associated with your user ID.



The technique you use to write to the database can also affect the speed of the operation. Code that processes data one row at a time, like the DATA step, will transfer data more slowly than utility procedures like PROC COPY and PROC APPEND, which transfer data in blocks.



Getting the smaller table into the database is a two-step process. First, I'll use a LIBNAME statement with the dbmstemp=yes option to assign a libref. The dbmstemp option requires a global connection to the database and allows you to create temporary tables. These tables won't show up in the SAS library, but can be used as long as the connection persists. When the last connection to the database is closed, temporary tables are automatically deleted. I prefer to use PROC APPEND to copy tables because it allows me to specify dataset options for the table being copied. To ensure that my target table doesn't already exist so that I don't inadvertently append to an existing table, I'll drop the table. I like using PROC FEDSQL for this because the FORCE option suppresses error and warning messages if the table doesn't exist. Finally, I'll load my Excel table to the database as a temporary table using PROC APPEND and the appropriate SAS/ACCESS interface data set options for bulk loading. This makes the data upload process much faster.

DEMO: Eliminating Cross-Library Operations

```
/*************************************************************************
****
 Eliminating Cross-Library Operations
*************************************************************************
***/
/* Step 1 - The cross-library join  */
libname xl xlsx "&path/data/marketing.xlsx";
options sastrace=',,,d' sastraceloc=saslog nostsuffix;

proc SQL;
create table cross_library as
select o.Customer_ID
      ,o.Customer_Name
      ,count(*) as Orders
      ,sum(total_sale) as Total_Spent
   from mydb.Orders o
      inner join
        xl.top100 t
      on o.Customer_ID=t.Customer_ID
```

```
   group by  o.Customer_ID
            ,o.Customer_Name
   order by o.Customer_ID
;
quit;

libname xl clear;
options sastrace=off;
/*************************************************************************
****
 Note: This cross-library join took about several seconds to complete.
*************************************************************************
***/

/*************************************************************************
****
 Step 2 - Eliminating the Cross-Library join
      2.a: Load the Excel table to temporary table space in the database.
Note: Many databases allow temporary tables. Examples include DB2, Hadoop,
      MS SQL Server, MySQL, Oracle, PostgreSQL, SAP HANA, SingleStore,
      Snowflake, and Teradata
*************************************************************************
***/
/* Access temporary table space in the database */
libname dbTemp mysql server="server.demo.sas.com" port=3306
               database=wwdv4 connection=global dbcommit=0 dbmstemp=yes
               user="&userID" password="&password";

libname xl xlsx "/home/student/Courses/etdp/data/marketing.xlsx";

/* Just in case the table already exists :-) */
proc FedSQL;
   drop table dbTemp.top100 force;
quit;

proc append base=dbTemp.top100(BL_DEFAULT_DIR="&path/data" bulkload=yes)
            data=xl.top100;
run;

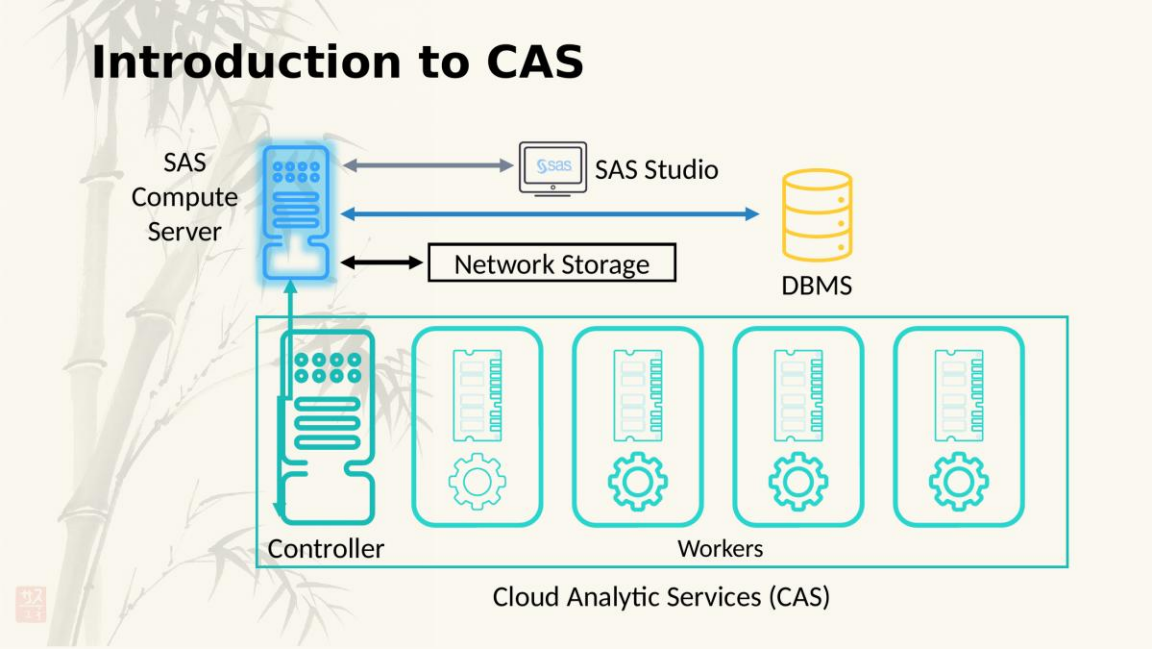
libname xl clear;

/*************************************************************************
****
 Note: The table upload took less than one second.
*************************************************************************
***/

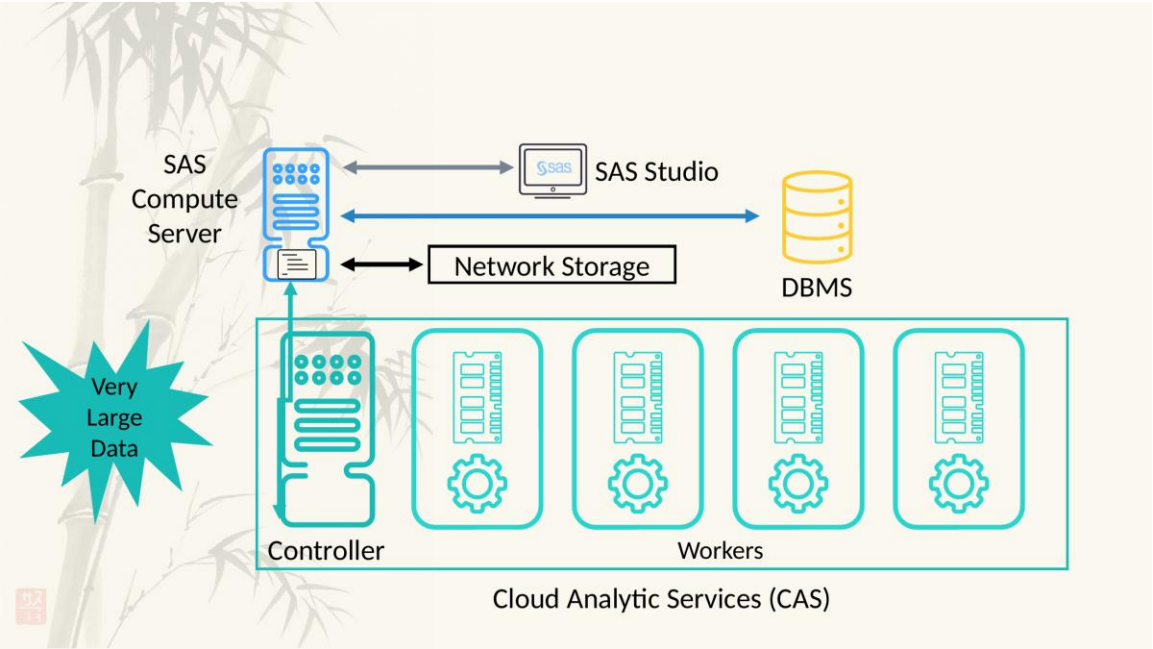
/*************************************************************************
****
 Step 2.b: Now the join can happen in the database.
*************************************************************************
***/
options sastrace=',,,d' sastraceloc=saslog nostsuffix;
title "Inner Join - Temp table";
proc SQL;
create table database_join as
select o.Customer_ID
      ,o.Customer_Name
```

```
      ,count(*) as Orders
      ,sum(total_sale) as Total_Spent
   from myDB.Orders o
      inner join
        dbTemp.top100 t
      on o.Customer_ID=t.Customer_ID
   group by  o.Customer_ID
           ,o.Customer_Name
   order by o.Customer_ID
;
quit;
title ;
options sastrace=off;
/*************************************************************************
****
 Note: This entire process ran in about half the time required for the
       cross-library join.
*************************************************************************
***/
```

INTRODUCTION TO CAS

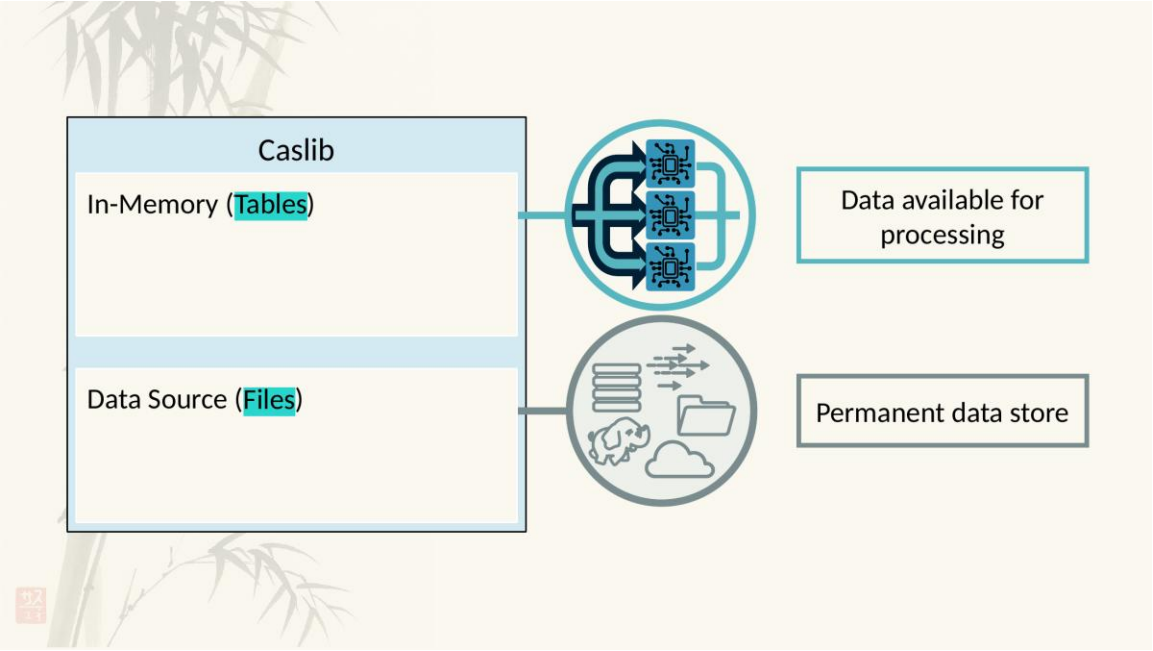


Up until now, we've been talking about running code on the SAS Compute Server. But Viya also includes Cloud Analytic Services, or CAS for short, which provides a native, massively parallel processing engine.

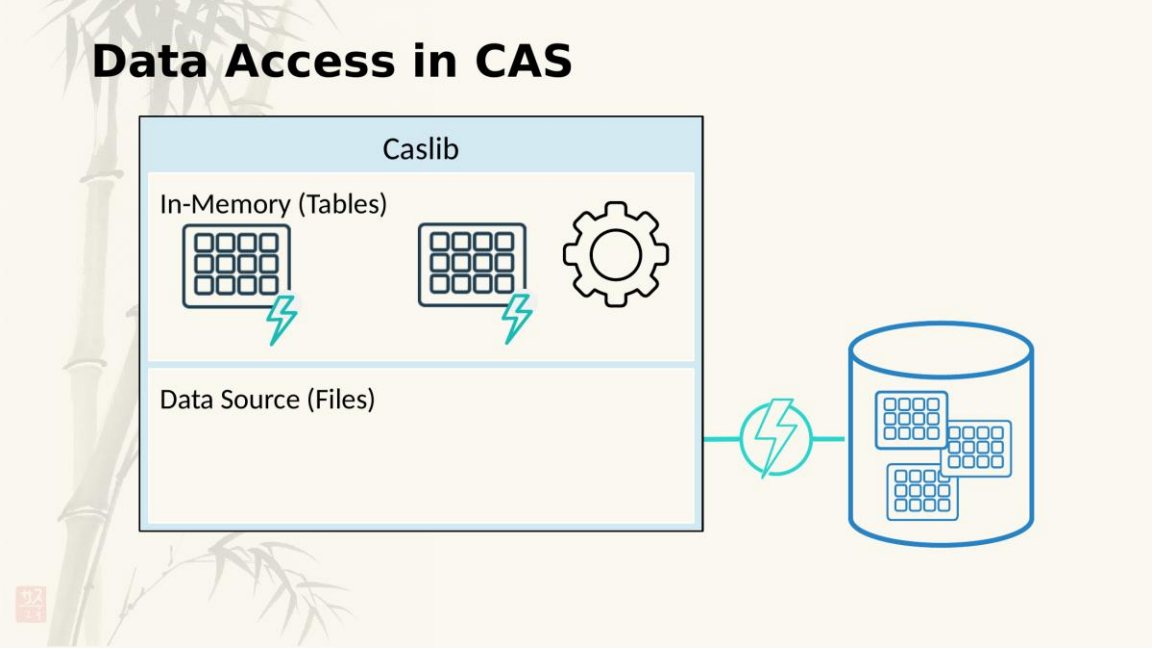


CAS offers some distinct advantages when manipulating large data with complex algorithms. CAS tables are pre-loaded into memory and distributed across the CAS workers. The Compute Server can send program code to CAS for processing. Workers process their portion of the table in parallel, significantly speeding up execution of computationally intense programs. Because tables persist in memory, subsequent program steps and multiple users can access the data without needing to reload the table, greatly reducing I/O delays.

INTRODUCTION TO CASLIBS

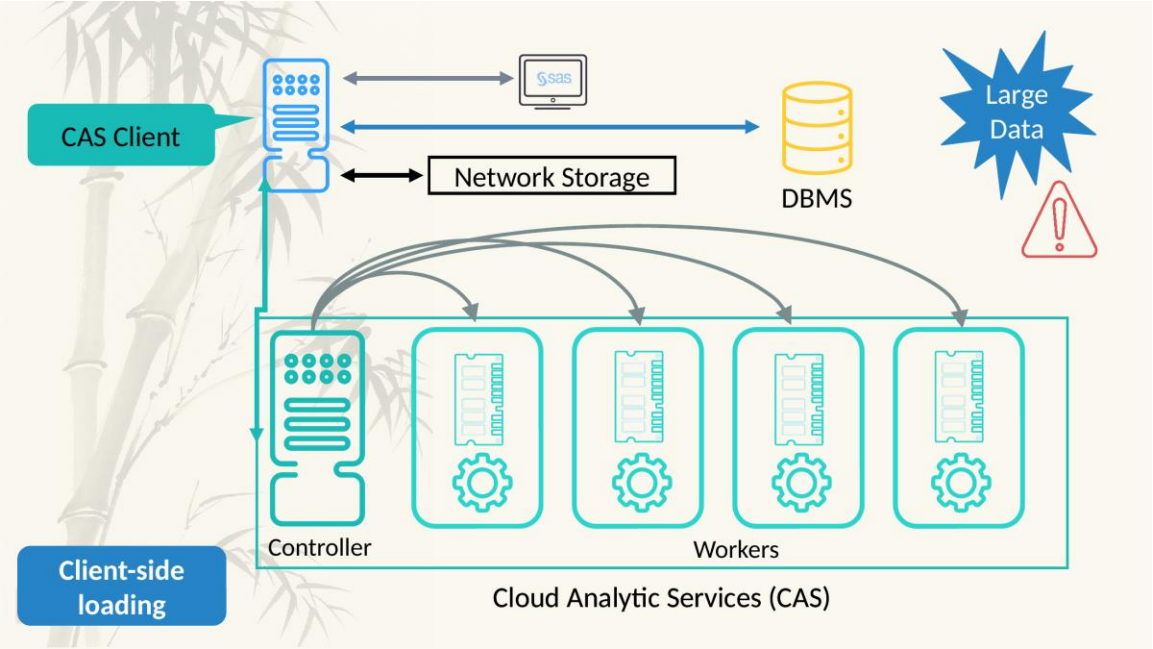


In CAS, all data is accessed via caslibs. A caslib is made up of two main parts. The data source portion contains the information necessary to connect to a permanent data storage location. Data can be stored as files on disk, in databases like Oracle and Teradata, or other systems like Hadoop or cloud-based sources. CAS connects to data sources using SAS Viya data connectors. Data stored in the data source of a caslib are referred to as "files". Files can't be processed directly in CAS. The other part of the caslib is the in-memory portion. In CAS, only in-memory data are available for processing. Data in the in-memory portion are referred to as "tables" or "CAS tables".



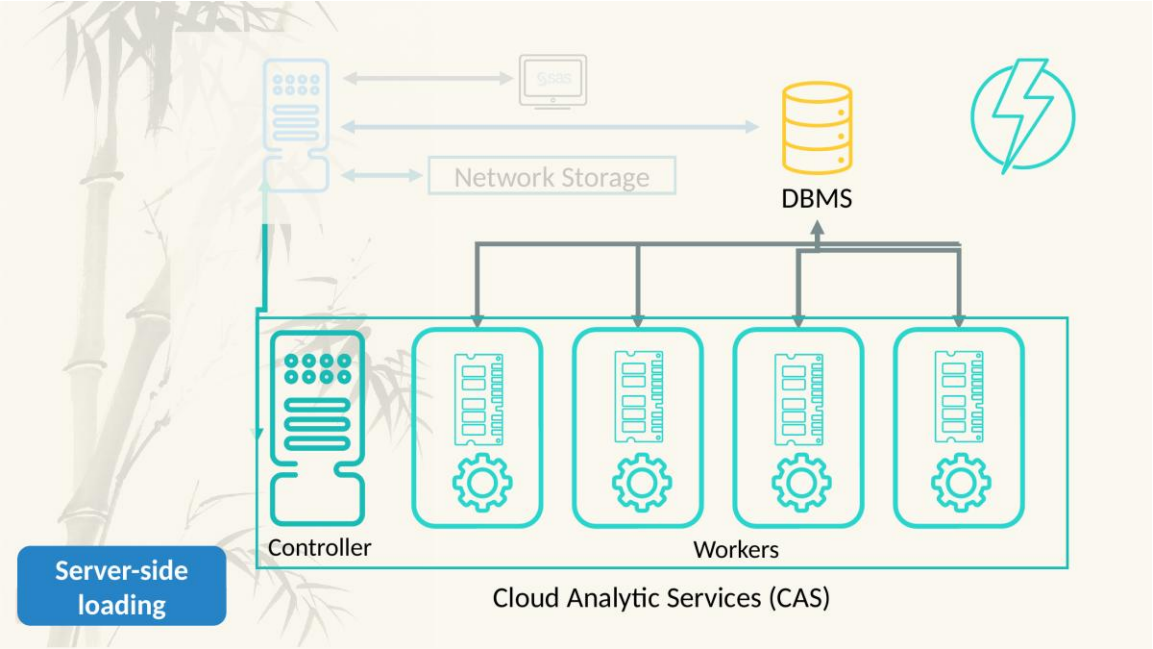
Now, when caslib in-memory data are referred to as tables and data in the data source are referred to as files, it can be hard to keep the terminology straight when the data source is a database. Database tables are justfiles as far as the caslib is concerned. Once a database table is copied to the in-memory portion of the caslib, it is processed just like any other CAS table.

Client-side Data Loading



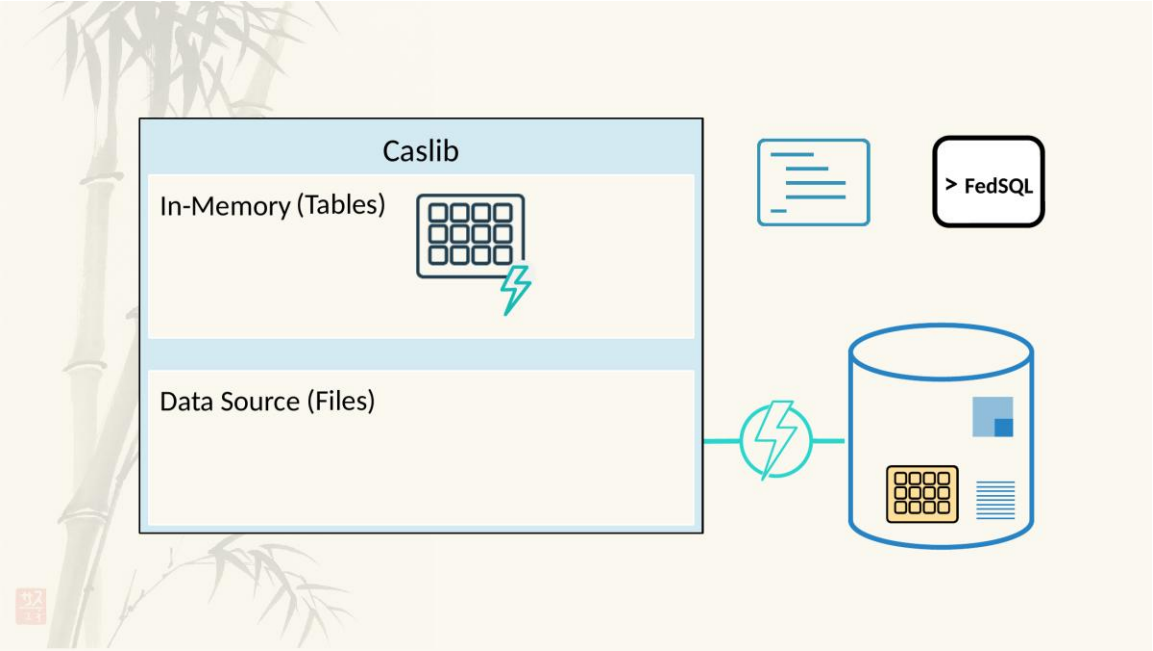
You can load data into CAS from the Compute server. But when loading database tables to a caslib this way, the data must first be extracted to the Compute Server, then sent to the CAS controller for distribution to the workers. Loading data to CAS this way is referred to as "client-side loading" because the data must first pass through the CAS client, in this case, the Compute Server, before being passed to CAS. Client-side loading can be quite slow when loading large tables and is even slower when the data is sourced from a database table because it must be transmitted over the network twice.

Server-Side Data Loading



CAS doesn't need the SAS Compute Server to access database tables. It can directly access data from database systems using Data Connector technology specifically designed for this massively parallel

processing environment. This is commonly referred to as "server-side loading", because only the CAS server is involved, and it's usually the fastest method for loading database data into CAS.



DATA step code can execute in CAS but can only process in-memory CAS tables. However, for FedSQL programs, SAS Viya data connectors provide both explicit and implicit pass-through to database data. This provides the ability to conduct subsetting and summarization operations in the database before loading the result as a CAS in-memory table.

DEMO: WORKING WITH DBMS DATA IN CAS

```
/*************************************************************************
****
 Server-side vs. Client-side Loading
 Step 1 - Viewing caslib contents from Compute Server
*************************************************************************
***/
/* Compare PROC CONTENTS results  */
title "Contents viewed from the Compute Server";
title2 "SAS libref with the DBMS schema as the data source";
proc contents data=mydb._all_ nods;
run;
title2 "Caslib with the DBMS schema as the data source";
proc contents data=myCASDB._all_ nods;
run;
title;

/*************************************************************************
****
 Notes for PROC CONTENTS (the view from the Compute Server)
 - PROC CONTENTS for the SAS library lists the names of the MySQL tables in
   the database schema, treating them as tables.
 - PROC CONTENTS for the caslib indicates no tables are loaded, treating
the
   tables in the database schema as files.
```

```
*************************************************************************
***/

/*************************************************************************
****
 Step 2 - Viewing caslib contents from CAS
*************************************************************************
***/
proc casutil;
title "Contents viewed from the Compute Server";
title2 "myCASDB caslib data source files";
   list files incaslib="myCASDB" DATASOURCEOPTIONS=(schema="wwdv4");
quit;
title;

/*************************************************************************
****
 Notes for PROC CASUTIL (the view from CAS)
  - DBMS tables are listed as files
  - Identifiers (names) are mixed case
  - MySQL identifiers are case sensitive
*************************************************************************
***/

/*************************************************************************
****
 Step 3 - Compare client-side vs. server-side load.
*************************************************************************
***/
/* Client-side load from a Compute Server SAS/ACCESS libref (DB) to caslib
in-memory table*/
proc casutil;
   load data=mydb.Mortgages
        casout="mortgages_client_side" outcaslib="myCASDB" replace;
quit;

/* Server-side load from the caslib myCASDB MySQL data source to in-memory
table */
proc casutil;
   load casdata="Mortgages"  incaslib="myCASDB"
         casout="mortgages_server_side" outcaslib="myCASDB" replace;
quit;

/*************************************************************************
****
 Note that the data loaded faster when loaded server-side (from the
 caslib data source) instead of client-side (from the SAS/ACCESS libref).

 Thumb rule:
 Load CAS tables from the caslib data source (server-side load) when
possible.
*************************************************************************
***
 Step 5 - A little clean up of caslib memory
*************************************************************************
***/
proc  datasets library=myCASDB nolist nodetails kill;
run; quit;
```

```
/*************************************************************************
****
 Notes
  - No tables are loaded in the caslib
  - As before, DBMS table are listed as "files"
*************************************************************************
***/
/*************************************************************************
****
 Step 8 - Apparent implicit passthrough with FedSQL
*************************************************************************
***/
title "Implicit Passthrough - CAS has no tables loaded";
proc fedsql sessref=conn iptrace _method;
select count(*)
   from myCASDB."Mortgages"
;
quit;

/*************************************************************************
****
 Note
 - This ran pretty fast, but not lightning speed.
 - IPTRACE messages appear as WARNINGs
 - IPTRACE indicates unable to fully push-down to the database.
 - _METHOD shows that the table was loaded to CAS, and aggregation
accomlished
   with CAS actions.
*************************************************************************
***/

/*************************************************************************
****
 Step 9 - Explicit passthrough with FedSQL
*************************************************************************
***/
title2 "Explicit query";
proc FedSQL sessref=conn iptrace;
select * from connection to myCASDB
   (select count(*) from wwdv4.Mortgages);
quit;
title;
/*************************************************************************
****
 Note Explicit ran 10x faster than the implicit query.
*************************************************************************
***/
/*************************************************************************
****
 Step 7 - Attempt implicit passthrough with DATA step
*************************************************************************
***/
data myCASDB.grade_a
     myCASDB.grade_b
     myCASDB.grade_c
     myCASDB.grade_d
     /sessref=conn
     ;
```

```
   set myCASDB."Mortgages"n;
   where loangrade in ('A','B','C','D');
   select (LOANGRADE);
      when ('A') output myCASDB.grade_a;
      when ('B') output myCASDB.grade_b;
      when ('C') output myCASDB.grade_c;
      when ('D') output myCASDB.grade_d;
   end;
run;

/*************************************************************************
****
 Note that the DATA step fails because the desired table is not in CAS
memory.
*************************************************************************
***/
/*************************************************************************
****
 Step 10 - Explicit FedSQL pass-through with DS2
*************************************************************************
***/
proc ds2 sessref=conn;
data myCASDB.grade_a
     myCASDB.grade_b
     myCASDB.grade_c
     myCASDB.grade_d
      /overwrite=yes;
   method run();
       set {select * from connection to myCASDB
                (select * from wwdv4.Mortgages where loangrade in
('A','B','C','D'))};
       select (LOANGRADE);
          when ('A') output myCASDB.grade_a;
          when ('B') output myCASDB.grade_b;
          when ('C') output myCASDB.grade_c;
          when ('D') output myCASDB.grade_d;
       end;
   end;
enddata;
run;
quit;

/*************************************************************************
****
 Step 11 - Loading a CAS table with FedSQL pass-through
*************************************************************************
***/
proc FedSQL sessref=conn;
drop table myCASDB.Summary force;
create table myCASDB.Summary as
select * from connection to myCASDB
(select LoanGrade
      ,cast(Count(*) as Double) as "Number"
      , sum(Amount) as "Total"
      , avg(amount) as "Avg"
    from wwdv4.Mortgages
    where year(LastPayment)=2022
    group by LoanGrade)
```

```
;
quit;
proc casutil;
title "myCASDB caslib tables (MySQL tables loaded in CAS memory)";
   list tables incaslib="myCASDB";
quit;
/*************************************************************************
****
 Note:
 - Original table is 6.6M rows, would take ~2 minutes to load to CAS
 - I only need the summary to work with in CAS
 - With pass-through, summarized data loaded to a CAS table in a few
seconds
*************************************************************************
***/
/* Working with CAS tables using Compute Server processes */
ods pdf file="&path/results/myreport.pdf" startpage=no;
title "Mortgage Totals by Loan Type";
proc sgplot data=myCASDB.Summary;
    vbar LoanGrade/response=Total;
    keylegend / location=inside position=topright;
run;
proc sql;
select     LoanGrade
          ,Number format=comma12.
          ,Avg format=dollar12.2
          ,Total format=dollar20.2
    from myCASDB.Summary
    order by LoanGrade
;
quit;
ods pdf close;
title;

/*************************************************************************
****
 Step 12 - Saving a CAS table as a database table (write permission
required)
*************************************************************************
***/
proc casutil;
    save casdata="Summary" incaslib="myCASDB"
           casout="MortgageSummary" outcaslib="myCASDB";
title2 "myCASDB caslib data source files";
    list files incaslib="myCASDB" DATASOURCEOPTIONS=(schema="wwdv4");
run;
```

CONTACT INFORMATION

Your comments and questions are valued and encouraged. Contact the author at:

Mark Jordan
Email: sas.jedi@gmail.com
X: @JediRetired
Download link:

Working with Database Data in SAS Viya

https://bit.ly/DBDataViya

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ® indicates USA registration. Other brand and product names are trademarks of their respective companies.